

GUIDE TECHNIQUE · DATA SCIENCE & BI

Netflix, miroir du monde ? Pipeline data de A à Z

De la donnée brute au dashboard : chargement, analyse, nettoyage, data warehouse et restitution — sur le catalogue Netflix. Deux implémentations BI : Python + PostgreSQL et Talend + PostgreSQL.

Python · pandas

PostgreSQL

Talend

Power BI

Étoile + pont

Storytelling

Document technique de l'atelier « Netflix, miroir du monde ? » · Séance de 3 h 30 en présentiel · Données : Netflix Titles (Kaggle, licence CC0).

0 Objectifs & indicateurs (KPI)

Ce que le projet doit produire, et les indicateurs qui guident l'analyse et le dashboard.

► Objectifs du projet

- **Construire un pipeline data complet** : ingestion → nettoyage → data warehouse → BI → récit.
- **Répondre à une question captivante** : Netflix est-il une vitrine américaine ou une plateforme mondiale ?
- **Livrer un dashboard décisionnel** et une histoire claire appuyée sur les données.

► Les KPI du tableau de bord

KPI	Définition	Pourquoi il compte
Nombre de titres	Volume total du catalogue (films + séries)	La taille de l'offre
Répartition Films / Séries (%)	Part de chaque type de contenu	La nature du catalogue
Titres ajoutés par année	Volume ajouté chaque année	La dynamique de croissance (pic 2019)
Top pays producteurs	Nombre de titres par pays principal	La géographie du contenu
Part USA vs international (%)	Contenu américain vs reste du monde	Le cœur du récit — la mondialisation
Top genres	Genres les plus fréquents (multi-valués)	L'orientation éditoriale
Répartition par classification	TV-MA, TV-14, R...	Le public visé
Durée moyenne	Minutes (films) / saisons (séries)	Le format type

8 797

titres au catalogue

69,7 %

de films (30,3 % séries)

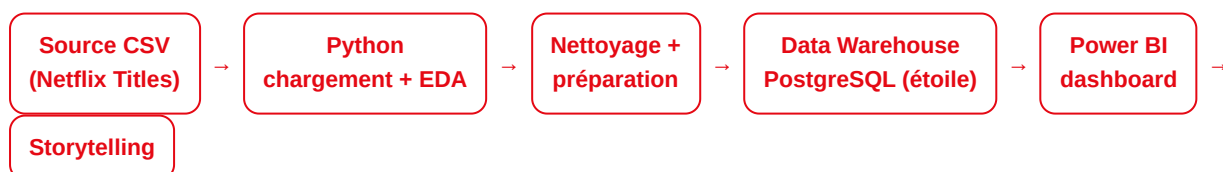
36 %

de contenu américain
seulement

2019

pic d'ajouts (2 016 titres)

► Architecture du projet



Deux chemins pour alimenter le DWH : la **VERSION A** en **Python** (pandas + SQLAlchemy) ; la **VERSION B** avec **Talend** (ETL visuel). Même destination PostgreSQL, même modèle.

1 Chargement des données

On charge le fichier des titres Netflix (une ligne par titre, 12 colonnes).

PYTHON

```
import pandas as pd

# Netflix Titles (Kaggle, CC0) : 1 ligne = 1 titre
df = pd.read_csv("netflix_titles.csv")
print(df.shape)          # (8807, 12)
print(df["type"].value_counts())  # Movie 6131 / TV Show 2676
df.head()
```

Colonnes clés : type (Movie/TV Show), title, country, date_added, release_year, rating, duration, listed_in (genres, multi-valués).

2 Analyse exploratoire (EDA)

On repère la structure, les trous et les colonnes « sales » à traiter.

PYTHON

```
df.info()                  # types + memoire
df.isna().sum()            # trous : director 2634, country 831, cast 825, rating 4...
df["listed_in"].head()     # "Dramas, International Movies" -> plusieurs genres par cellule
df["duration"].head()      # "90 min" (film) ou "2 Seasons" (serie) -> texte a decouper
df["date_added"].head()    # "September 25, 2021" -> texte a convertir en date
```

Trois pièges à repérer : date_added est du texte, duration mélange minutes et saisons, et listed_in / country contiennent *plusieurs valeurs* dans une seule cellule.

3 Nettoyage des données

On convertit les dates, on comble les trous, on isole le pays principal.

PYTHON

```
clean = df.copy()

# 1. date_added (texte) -> vraie date, puis on retire les 10 lignes sans date
clean["date_added"] = pd.to_datetime(clean["date_added"].str.strip(),
                                     format="%B %d, %Y", errors="coerce")
clean = clean.dropna(subset=["date_added"])

# 2. Combler les trous non bloquants
clean["country"] = clean["country"].fillna("Unknown")
clean["rating"] = clean["rating"].fillna("Not Rated")

# 3. Pays principal = 1er pays liste (la cellule peut en contenir plusieurs)
clean["primary_country"] = clean["country"].str.split(",").str[0].str.strip()

# 4. Doublons de titres
```

```
clean = clean.drop_duplicates(subset=["title", "type", "release_year"])
print(clean.shape)           # (8797, ...)
```

4 Préparation & variables calculées

On dérive l'année/mois d'ajout et on découpe la durée en valeur + unité.

PYTHON

```
clean["year_added"] = clean["date_added"].dt.year
clean["month_added"] = clean["date_added"].dt.month

# duration : "90 min" / "2 Seasons" -> nombre + unite
import numpy as np
clean["duration"] = clean["duration"].fillna("0")
clean["dur_value"] = clean["duration"].str.extract(r"(\d+)").astype(float)
clean["dur_unit"] = np.where(clean["type"]=="Movie", "min", "saisons")
```

5 Conception du Data Warehouse

Un modèle en étoile — enrichi d'une **table de pont** car un titre a plusieurs genres.

► Le grain

La table de faits `fact_titles` a pour grain un **titre**. Chaque titre pointe vers son type, son pays, sa date d'ajout et sa classification.

► Le many-to-many des genres

Un titre appartient à **plusieurs genres**. On ne peut pas mettre ça dans une colonne : on crée `dim_genre` + une **table de pont** `bridge_title_genre`.

`dim_type`

`dim_country`

`fact_titles`

`dim_date`

`dim_rating`

+ `dim_genre ↔ pont`

SQL (POSTGRESQL)

```
-- DDL PostgreSQL : étoile + pont pour les genres
CREATE TABLE dim_type (type_key INT PRIMARY KEY, type VARCHAR(10));
CREATE TABLE dim_country (country_key SERIAL PRIMARY KEY, country_name VARCHAR(100));
CREATE TABLE dim_date (date_key SERIAL PRIMARY KEY, year_added INT, month_added INT);
CREATE TABLE dim_rating (rating_key SERIAL PRIMARY KEY, rating VARCHAR(20));
CREATE TABLE dim_genre (genre_key SERIAL PRIMARY KEY, genre VARCHAR(60));

CREATE TABLE fact_titles (
  show_id VARCHAR(10) PRIMARY KEY,
  type_key INT REFERENCES dim_type(type_key),
  country_key INT REFERENCES dim_country(country_key),
  date_key INT REFERENCES dim_date(date_key),
  rating_key INT REFERENCES dim_rating(rating_key),
  release_year INT,
  dur_value NUMERIC, -- minutes (film) ou nb de saisons (serie)
  dur_unit VARCHAR(10)
);

-- table de pont : plusieurs genres par titre
CREATE TABLE bridge_title_genre (
  show_id VARCHAR(10) REFERENCES fact_titles(show_id),
  genre_key INT REFERENCES dim_genre(genre_key),
  PRIMARY KEY (show_id, genre_key)
);
```

6 Implémentation du DWH

Deux versions au choix, même résultat dans PostgreSQL.

VERSION A Python + PostgreSQL

PYTHON

```
from sqlalchemy import create_engine

# --- DIMENSIONS ---
dim_type = pd.DataFrame({"type_key": [1, 2], "type": ["Movie", "TV Show"]})
dim_country = (clean[["primary_country"]].drop_duplicates().reset_index(drop=True)
               .reset_index().rename(columns={"index": "country_key",
                                               "primary_country": "country_name"}))
dim_date = (clean[["year_added", "month_added"]].drop_duplicates()
            .sort_values(["year_added", "month_added"]).reset_index(drop=True)
            .reset_index().rename(columns={"index": "date_key"}))
dim_rating = (clean[["rating"]].drop_duplicates().reset_index(drop=True)
              .reset_index().rename(columns={"index": "rating_key"}))

# dim_genre : on "explode" listed_in (1 ligne par genre)
g = clean[["show_id", "listed_in"]].copy()
g["genre"] = g["listed_in"].str.split(",")
g = g.explode("genre"); g["genre"] = g["genre"].str.strip()
dim_genre = (g[["genre"]].drop_duplicates().reset_index(drop=True)
             .reset_index().rename(columns={"index": "genre_key"}))

# --- FAITS (on remplace les libelles par les cles) ---
fact = (clean.merge(dim_type, on="type")
        .merge(dim_country, left_on="primary_country", right_on="country_name")
        .merge(dim_date, on=["year_added", "month_added"])
        .merge(dim_rating, on="rating"))
fact = fact[["show_id", "type_key", "country_key", "date_key", "rating_key",
            "release_year", "dur_value", "dur_unit"]]

# --- PONT titre <-> genre ---
bridge = g.merge(dim_genre, on="genre")[["show_id", "genre_key"]]

# --- CHARGEMENT ---
engine = create_engine("postgresql+psycopg2://user:pwd@localhost:5432/netflix_dwh")
for name, d in [("dim_type", dim_type), ("dim_country", dim_country), ("dim_date", dim_date),
               ("dim_rating", dim_rating), ("dim_genre", dim_genre),
               ("fact_titles", fact), ("bridge_title_genre", bridge)]:
    d.to_sql(name, engine, if_exists="append", index=False)
print("Charge :", len(fact), "faits,", len(bridge), "liens genre")
```

VERSION B Talend + PostgreSQL

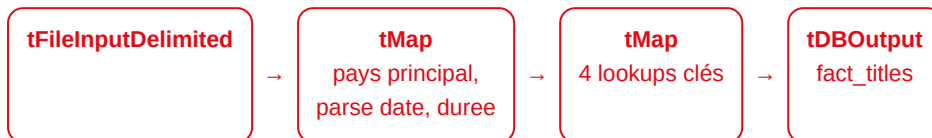
Même logique dans Talend Open Studio. Les **dimensions d'abord**, puis les faits et le pont. Deux composants Talend sont la clé ici : **tNormalize** (éclater les genres) et **tMap** (jointures lookup).

► Jobs de dimensions

- dim_type** : tFixedFlowInput (2 lignes Movie/TV Show) → tDBOutput .
- dim_country** : tFileInputDelimited → tMap (pays principal via StringHandling.LEFT jusqu'à la 1^{re} virgule) → tUniqRow → tDBOutput .

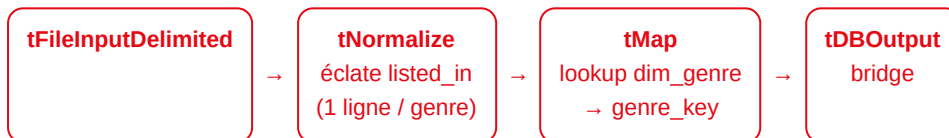
- **dim_date** : `tFileInputDelimited` → `tMap` (`TalendDate.parseDate("MMMM d, yyyy", row.date_added)` , extraire année/mois) → `tUniqRow` → `tDBOutput` .
- **dim_rating** : `tFileInputDelimited` → `tUniqRow` (sur rating) → `tDBOutput` .
- **dim_genre** : `tFileInputDelimited` → `tNormalize` (colonne `listed_in` , séparateur « , ») → `tMap` (trim) → `tUniqRow` → `tDBOutput` .

► Job — Charger `fact_titles`



Dans le second **tMap**, quatre entrées *lookup* depuis PostgreSQL : `dim_type` (sur type), `dim_country` (sur pays principal), `dim_date` (sur année+mois), `dim_rating` (sur rating) → on récupère les 4 clés.

► Job — Charger le pont `bridge_title_genre`



Ordre impératif : toutes les dimensions avant `fact_titles` et le pont. On orchestre avec un job maître (**tRunJob** + déclencheurs *OnSubjobOk*).

7 Dashboarding avec Power BI

On connecte Power BI au data warehouse et on construit les visuels autour des KPI.

► Connexion & modèle

- **Obtenir les données** → PostgreSQL : base `netflix_dwh`, mode Import.
- Relations : `fact_titles` vers `dim_type`, `dim_country`, `dim_date`, `dim_rating` (plusieurs-à-un).
- Genres : `fact_titles` → `bridge_title_genre` → `dim_genre` (relation via le pont, filtre bidirectionnel sur le pont).

► Mesures DAX

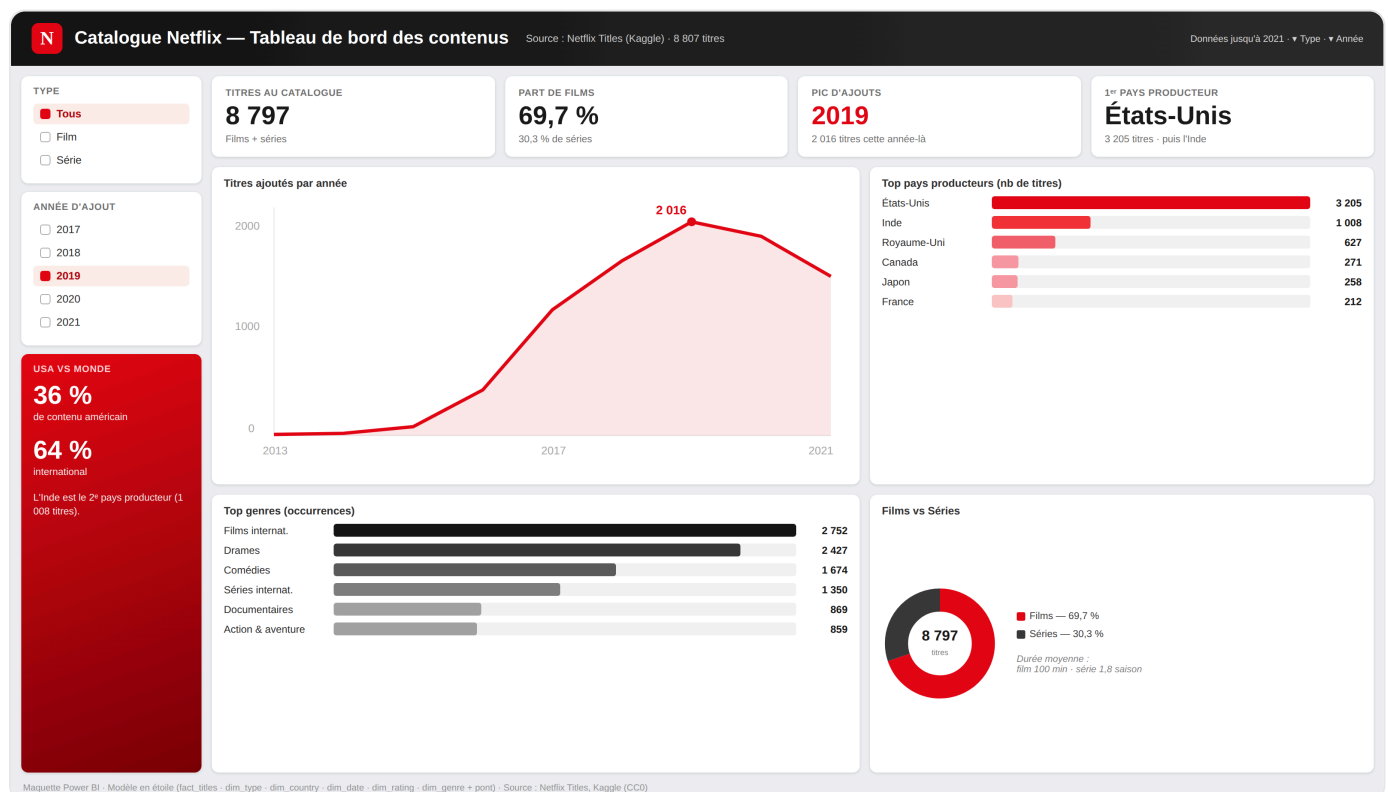
SQL (POSTGRESQL)

```
Nb titres = DISTINCTCOUNT(fact_titles[show_id])
Part de films = DIVIDE(CALCULATE([Nb titres], dim_type[type]="Movie"), [Nb titres])
Part USA = DIVIDE(CALCULATE([Nb titres], dim_country[country_name]="United States"), [Nb titres])
Duree moy film = CALCULATE(AVERAGE(fact_titles[dur_value]), fact_titles[dur_unit]="min")
```

► Visuels (correspondance avec la maquette)

- **4 cartes KPI** : nb titres, part de films, pic d'ajouts, 1^{er} pays.
- **Courbe / aire** : titres par `year_added` (la croissance).
- **Barres** : top pays producteurs · **barres** : top genres (via le pont).
- **Anneau** : Films vs Séries · **segments** : type, année, pays.
- **Carte focus** : part USA vs international.

► Maquette du dashboard cible



8 Storytelling — le récit des données

Le fil « Constat → Preuve → Message » à raconter en clôture.

► Le constat

Le catalogue Netflix a **explosé** en quelques années : de 11 titres ajoutés en 2013 à un pic de **2 016 en 2019**, avant un léger repli — la fin de la course au volume.

► Les preuves

- Le catalogue est surtout composé de **films (70 %)**, mais les séries montent.
- Surtout : **seulement 36 % des titres sont américains**. Le reste — 64 % — vient du monde entier, avec **l'Inde en 2^e position** (1 008 titres).
- Les genres dominants le confirment : « Films internationaux » et « Séries internationales » sont en tête.

► Le message

Netflix n'est plus une vitrine américaine : c'est devenu une **plateforme mondiale**, et la donnée le prouve chiffres à l'appui. La question « d'où vient ce que je regarde ? » a une réponse de plus en plus internationale.

► Le focus qui parle au public

Le catalogue est aussi très **récent** : 71 % des titres sont sortis après 2015, et l'écart médian entre la sortie d'un titre et son ajout n'est que d'**un an**. Netflix mise sur la fraîcheur, pas sur le fonds de catalogue.

Conseil d'animation : garde le « 36 % seulement d'américain » pour le milieu de séance — c'est le chiffre contre-intuitif qui accroche tout le monde.